

Spring Boot 3

Java 21

H2 / MySQL

JWT

Maven

1. Descripción General del Proyecto

En este proyecto desarrollarás una **API REST completa** con Java y Spring Boot para gestionar el catálogo y los préstamos de una biblioteca. El objetivo es aplicar la arquitectura en capas **Entidad** → **Repositorio** → **Servicio** → **Controlador**, persistencia con JPA/Hibernate, una base de datos relacional, y autenticación/autorización mediante **Spring Security + JWT**.

La aplicación expone endpoints REST que permitan registrar libros, autores y usuarios, gestionar préstamos y controlar el acceso según el rol de cada usuario (**ADMIN** o **USER**).

Aspecto	Detalle
Lenguaje	Java 21
Framework	Spring Boot 3.x (Spring Web, Spring Data JPA, Spring Security)
Base de datos	H2 (desarrollo) / MySQL o PostgreSQL (producción)
Autenticación	JWT (JSON Web Token) – sin estado (stateless)
Build tool	Maven
Formato de datos	JSON
Documentación API	Springdoc OpenAPI / Swagger UI (opcional, +puntos)

Nota: No se corregirá ningún proyecto que no compile o que no arranque con `mvn spring-boot:run` sin errores.

2. Estructura del Proyecto

El proyecto Maven se llamará **gestion-biblioteca** y tendrá la siguiente estructura de paquetes:

```
com.biblioteca ■■■ GestionBibliotecaApplication.java (clase principal) ■■■ config/ ■ ■■■
SecurityConfig.java ■ ■■■ JwtConfig.java ■■■ security/ ■ ■■■ JwtUtil.java ■ ■■■
JwtAuthenticationFilter.java ■ ■■■ UserDetailsServiceImpl.java ■■■ entity/ ■ ■■■ Autor.java
■ ■■■ Libro.java ■ ■■■ Usuario.java ■ ■■■ Prestamo.java ■■■ repository/ ■ ■■■
AutorRepository.java ■ ■■■ LibroRepository.java ■ ■■■ UsuarioRepository.java ■ ■■■
PrestamoRepository.java ■■■ service/ ■ ■■■ AutorService.java ■ ■■■ LibroService.java ■ ■■■
UsuarioService.java ■ ■■■ PrestamoService.java ■■■ controller/ ■ ■■■ AuthController.java ■
■■■ AutorController.java ■ ■■■ LibroController.java ■ ■■■ UsuarioController.java ■ ■■■
PrestamoController.java ■■■ dto/ ■ ■■■ LoginRequest.java ■ ■■■ LoginResponse.java ■■■
LibroDTO.java ■■■ PrestamoDTO.java
```

3. Modelo de Base de Datos

El diagrama entidad-relación contempla las siguientes tablas. JPA generará el esquema automáticamente a partir de las entidades (`spring.jpa.hibernate.ddl-auto=update`).

Tabla	Campos principales	Relaciones
autores	id (PK), nombre, apellidos, nacionalidad, fecha_nacimiento	1 autor → N libros
libros	id (PK), isbn (UNIQUE), titulo, genero, anio_publicacion, stock, autor_id (FK)	N libros → 1 autor N libros ↔ N préstamos
usuarios	id (PK), username (UNIQUE), password (BCrypt), email, rol (ADMIN/USER), activo	1 usuario → N préstamos
prestamos	id (PK), fecha_inicio, fecha_fin_prevista, fecha_devolucion, estado, usuario_id (FK), libro_id (FK)	N préstamos → 1 usuario N préstamos → 1 libro

Nota: El campo **estado** del préstamo puede ser: **ACTIVO**, **DEVUELTO**, **VENCIDO**. Modévalo como **@Enumerated(EnumType.STRING)**.

4. Capa de Entidades (entity)

Cada clase entidad representa una tabla de la base de datos. Debes anotar correctamente con JPA y aplicar las validaciones de Bean Validation (**jakarta.validation**).

4.1 Autor.java

Atributos y restricciones:

Campo	Tipo Java	Anotaciones / Validaciones
id	Long	@Id, @GeneratedValue(IDENTITY)
nombre	String	@NotBlank, @Size(min=2, max=80)
apellidos	String	@NotBlank, @Size(min=2, max=100)
nacionalidad	String	@Size(max=60)
fechaNacimiento	LocalDate	@Past (debe ser fecha pasada)
libros	List<Libro>	@OneToMany(mappedBy="autor", cascade=ALL)

4.2 Libro.java

Campo	Tipo Java	Anotaciones / Validaciones
id	Long	@Id, @GeneratedValue(IDENTITY)
isbn	String	@NotBlank, @Column(unique=true), @Pattern(regexp ISBN-13)
titulo	String	@NotBlank, @Size(min=1, max=200)
genero	String	@Size(max=60)

Campo	Tipo Java	Anotaciones / Validaciones
anioPublicacion	Integer	@Min(1450) @Max(año actual)
stock	int	@Min(0) – copias disponibles
autor	Autor	@ManyToOne @JoinColumn(name="autor_id") @NotNull

4.3 Usuario.java

Esta entidad implementa **UserDetails** de Spring Security o bien tienes la opción de crear un **UserDetailsService** aparte.

Campo	Tipo Java	Anotaciones / Validaciones
id	Long	@Id, @GeneratedValue(IDENTITY)
username	String	@NotBlank, @Column(unique=true), @Size(min=4, max=40)
password	String	@NotBlank (almacenada en BCrypt)
email	String	@Email, @NotBlank, @Column(unique=true)
rol	Rol (enum)	@Enumerated(STRING) – valores: ADMIN, USER
activo	boolean	true por defecto

4.4 Prestamo.java

Campo	Tipo Java	Anotaciones / Validaciones
id	Long	@Id, @GeneratedValue(IDENTITY)
fechaInicio	LocalDate	Asignada automáticamente (LocalDate.now())
fechaFinPrevista	LocalDate	@FutureOrPresent – mínimo fechaInicio + 7 días
fechaDevolucion	LocalDate	Null si no ha sido devuelto
estado	EstadoPrestamo	@Enumerated(STRING): ACTIVO, DEVUELTO, VENCIDO
usuario	Usuario	@ManyToOne @JoinColumn(name="usuario_id")
libro	Libro	@ManyToOne @JoinColumn(name="libro_id")

5. Capa de Repositorios (repository)

Cada repositorio extiende **JpaRepository<Entidad, Long>**. Spring Data JPA generará la implementación automáticamente. Debes añadir los métodos de consulta personalizados que se indican.

Repositorio	Métodos personalizados requeridos
AutorRepository	Optional<Autor> findByNameAndApellidos(String nombre, String apellidos); List<Autor> findByNacionalidad(String nacionalidad);

Repositorio	Métodos personalizados requeridos
LibroRepository	Optional<Libro> findByIsbn(String isbn); List<Libro> findByTituloContainingIgnoreCase(String titulo); List<Libro> findByAutorId(Long autorId); List<Libro> findByStockGreaterThan(int stock);
UsuarioRepository	Optional<Usuario> findByUsername(String username); Optional<Usuario> findByEmail(String email); List<Usuario> findByRol(Rol rol);
PrestamoRepository	List<Prestamo> findByUsuariId(Long usuariId); List<Prestamo> findByLibroId(Long libroId); List<Prestamo> findByEstado(EstadoPrestamo estado); List<Prestamo> findByFechaFinPrevistaBeforeAndEstado(LocalDate fecha, EstadoPrestamo estado); // préstamos vencidos

Tip: Spring Data JPA deriva las consultas SQL a partir del nombre del método. Si el método es complejo, usa @Query con JPQL o SQL nativo.

6. Capa de Servicios (service)

Los servicios contienen la **lógica de negocio**. Cada clase lleva la anotación **@Service** e inyecta su repositorio con **@Autowired** o mediante constructor. Los métodos que modifican datos llevan **@Transactional**.

6.1 AutorService

Método	Descripción y reglas de negocio
List<Autor> listarTodos()	Devuelve todos los autores ordenados por apellidos.
Autor buscarPorId(Long id)	Lanza ResourceNotFoundException si no existe.
Autor crear(Autor autor)	Valida que no exista ya un autor con mismo nombre y apellidos.
Autor actualizar(Long id, Autor datos)	Actualiza campos permitidos. Lanza excepción si no existe.
void eliminar(Long id)	Solo puede eliminarse si no tiene libros asociados; en caso contrario lanza IllegalStateException.
List<Autor> buscarPorNacionalidad(String nacionalidad)	Filtra por nacionalidad (ignora mayúsculas).

6.2 LibroService

Método	Descripción y reglas de negocio
List<Libro> listarTodos()	Devuelve todos los libros.
Libro buscarPorIsbn(String isbn)	Lanza ResourceNotFoundException si no existe.
Libro crear(Libro libro)	Valida que el ISBN no esté duplicado. Valida que el autor exista en la BD.

Método	Descripción y reglas de negocio
Libro actualizar(Long id, Libro datos)	Actualiza título, género, año, stock. No permite cambiar ISBN.
void eliminar(Long id)	No se puede eliminar si tiene préstamos activos.
List<Libro>; buscarPorTitulo(String titulo)	Búsqueda parcial e insensible a mayúsculas.
List<Libro>; listarDisponibles()	Stock > 0.

6.3 PrestamoService

Método	Descripción y reglas de negocio
Prestamo crear(Long usuarioid, Long libroid, LocalDate fechaFinPrevista)	Comprueba que el libro tenga stock > 0. Reduce stock del libro en 1. Crea préstamo con estado ACTIVO.
Prestamo devolver(Long prestamoid)	Establece fechaDevolucion = hoy. Estado = DEVUELTO. Aumenta stock del libro en 1.
List<Prestamo>; listarPorUsuario(Long usuarioid)	Historico de préstamos de un usuario.
List<Prestamo>; listarVencidos()	Estado ACTIVO con fechaFinPrevista < hoy. Cambia su estado a VENCIDO.
List<Prestamo>; listarActivos()	Todos los préstamos con estado ACTIVO.

6.4 UsuarioService

Método	Descripción y reglas de negocio
Usuario registrar(Usuario usuario)	Codifica la password con BCryptPasswordEncoder. Valida que username y email sean únicos.
Usuario buscarPorUsername(String username)	Usado por Spring Security.
void cambiarPassword(Long id, String nuevaPassword)	Valida nueva contraseña (min 8 chars, 1 mayúscula, 1 dígito, 1 especial). Codifica y guarda.
void desactivar(Long id)	Pone activo=false. Solo ADMIN puede hacerlo.
List<Usuario>; listarTodos()	Solo accesible para ADMIN.

7. Capa de Controladores (controller)

Cada controlador lleva **@RestController** y **@RequestMapping**. Los métodos devuelven **ResponseEntity<?>** con el código HTTP adecuado. Usa **@Valid** para activar las validaciones de Bean Validation en los DTO/entidades.

7.1 AuthController – /api/auth

Método HTTP	Endpoint	Acceso	Descripción
POST	/register	Público	Registra un nuevo usuario con rol USER.
POST	/login	Público	Autentica credenciales y devuelve JWT.

Cuerpo de /login (JSON):

```
{ "username": "maria", "password": "Secreta1!" }
```

Respuesta de /login (JSON):

```
{ "token": "eyJhbGc...", "tipo": "Bearer", "username": "maria", "rol": "USER" }
```

7.2 AutorController – /api/autores

Método	Endpoint	Rol	Respuesta
GET	/	USER, ADMIN	200 + lista de autores
GET	//{id}	USER, ADMIN	200 + autor o 404
POST	/	ADMIN	201 + autor creado
PUT	//{id}	ADMIN	200 + autor actualizado o 404
DELETE	//{id}	ADMIN	204 o 409 si tiene libros
GET	/nacionalidad/{n}	USER, ADMIN	200 + lista filtrada

7.3 LibroController – /api/libros

Método	Endpoint	Rol	Respuesta
GET	/	USER, ADMIN	200 + lista de libros
GET	//{id}	USER, ADMIN	200 + libro o 404
GET	/isbn/{isbn}	USER, ADMIN	200 + libro o 404
GET	/buscar?titulo=X	USER, ADMIN	200 + lista filtrada
GET	/disponibles	USER, ADMIN	200 + libros con stock > 0
POST	/	ADMIN	201 + libro creado
PUT	//{id}	ADMIN	200 + libro actualizado o 404
DELETE	//{id}	ADMIN	204 o 409 si tiene préstamos activos

7.4 PrestamoController – /api/prestamos

Método	Endpoint	Rol	Respuesta
POST	/	USER, ADMIN	201 + préstamo creado o 409 sin stock
PUT	//{id}/devolver	USER, ADMIN	200 + préstamo devuelto o 404

Método	Endpoint	Rol	Respuesta
GET	/usuario/{id}	USER, ADMIN	200 + préstamos del usuario
GET	/activos	ADMIN	200 + todos los activos
GET	/vencidos	ADMIN	200 + vencidos actualizados

7.5 UsuarioController – /api/usuarios

Método	Endpoint	Rol	Respuesta
GET	/	ADMIN	200 + lista de usuarios
GET	/id	ADMIN	200 + usuario o 404
PUT	/id/password	ADMIN, propio USER	200 + ok o 400
PUT	/id/desactivar	ADMIN	200 + usuario desactivado

8. Seguridad – Spring Security + JWT

La autenticación es **sin estado (stateless)**: el servidor no guarda sesión. Cada petición debe incluir el token JWT en la cabecera **Authorization: Bearer <token>**.

8.1 Dependencias Maven (pom.xml)

```
<dependency> <groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-security</artifactId> </dependency> <dependency>
<groupId>io.jsonwebtoken</groupId> <artifactId>jjwt-api</artifactId>
<version>0.11.5</version> </dependency> <dependency> <groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-impl</artifactId> <version>0.11.5</version> <scope>runtime</scope>
</dependency> <dependency> <groupId>io.jsonwebtoken</groupId>
<artifactId>jjwt-jackson</artifactId> <version>0.11.5</version> <scope>runtime</scope>
</dependency>
```

8.2 Clases de seguridad a implementar

Clase	Responsabilidad
JwtUtil	Genera el token (generateToken), extrae el username (extractUsername), valida el token (validateToken). Usa una clave secreta configurada en application.properties.
JwtAuthenticationFilter	Extiende OncePerRequestFilter. Intercepta cada petición, extrae el JWT de la cabecera, lo valida y carga el usuario en el SecurityContext.
UserDetailsServiceImpl	Implementa UserDetailsService. El método loadUserByUsername busca el usuario en BD y lanza UsernameNotFoundException si no existe.
SecurityConfig	Clase @Configuration @EnableWebSecurity. Define la cadena de filtros: desactiva CSRF, sesión STATELESS, permite /api/auth/** sin autenticar, protege el resto con hasRole("ADMIN") o authenticated() según endpoint.

8.3 Flujo de autenticación

Paso	Descripción
1. POST /api/auth/login	El cliente envía username y password en JSON.
2. AuthController	Llama a AuthenticationManager.authenticate().
3. UserDetailsServiceImpl	Carga el usuario de la BD.
4. Credenciales correctas	JwtUtil.generateToken() crea el JWT firmado (HS256).
5. Respuesta	Se devuelve el token al cliente (200 OK).
6. Petición protegida	El cliente incluye: Authorization: Bearer <token>.
7. JwtAuthenticationFilter	Extrae y valida el token; inyecta Authentication en SecurityContext.
8. Controlador	Procesa la petición según el rol del usuario autenticado.

8.4 Configuración (application.properties)

```
# Base de datos H2 (desarrollo) spring.datasource.url=jdbc:h2:mem:biblioteca
spring.datasource.driver-class-name=org.h2.Driver spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true # JWT
jwt.secret=MiclaveSecretaSuperSegura1234567890!! jwt.expiration=86400000 # 24 horas en ms #
Jackson spring.jackson.serialization.write-dates-as-timestamps=false
```

9. DTOs (Data Transfer Objects)

Los DTO evitan exponer directamente las entidades JPA al exterior y permiten controlar qué campos se reciben y se envían. Se recomienda usar **Records** de Java 21 o clases con Lombok (**@Data**).

DTO	Campos	Uso
LoginRequest	String username String password	Recibir credenciales en POST /login
LoginResponse	String token String tipo ("Bearer") String username String rol	Respuesta de /login con el JWT
LibroDTO	Long id, String isbn, String titulo, String genero, int anioPublicacion, int stock, Long autorId, String autorNombre	Respuesta al consultar libros (incluye nombre del autor sin exponer entidad)
PrestamoDTO	Long id, LocalDate fechaInicio, LocalDate fechaFinPrevista, LocalDate fechaDevolucion, String estado, String usernameUsuario, String tituloLibro	Respuesta al consultar préstamos

Tip: Puedes usar **ModelMapper** o **MapStruct** para convertir entre entidades y DTOs, o hacerlo manualmente en los servicios.

10. Gestión Global de Excepciones

Crea una clase **GlobalExceptionHandler** anotada con **@RestControllerAdvice** para manejar excepciones de forma centralizada y devolver respuestas JSON consistentes.

Excepción	Código HTTP	Cuándo lanzarla
ResourceNotFoundException (custom)	404 Not Found	Entidad no encontrada por id, isbn, username, etc.
IllegalStateException	409 Conflict	Eliminar autor con libros, libro sin stock, préstamo ya devuelto.
MethodArgumentNotValidException	400 Bad Request	Fallo de validación @Valid en el cuerpo de la petición.
UsernameNotFoundException	401 Unauthorized	Usuario no encontrado al autenticar.
AccessDeniedException	403 Forbidden	Usuario autenticado sin el rol requerido.

Excepción	Código HTTP	Cuándo lanzarla
Exception (genérica)	500 Internal Server Error	Cualquier error no controlado.

Formato de respuesta de error (JSON):

```
{ "timestamp": "2024-11-15T10:30:00", "status": 404, "error": "Not Found", "message": "Libro con id 42 no encontrado", "path": "/api/libros/42" }
```

11. Datos Iniciales (DataLoader)

Crea una clase **DataLoader** que implemente **CommandLineRunner** y se anote con **@Component**. Al arrancar la aplicación insertará datos de prueba solo si la BD está vacía.

Dato	Cantidad mínima
Autores	5 autores con distintas nacionalidades
Libros	10 libros (al menos 2 por autor), con stock entre 1 y 5
Usuarios	1 ADMIN (admin / Admin1234!) y 3 USER
Préstamos	3 activos, 2 devueltos, 1 vencido

12. Criterios de Calificación

Apartado	Puntos	Detalles
Entidades y base de datos	1,5	Mapeo JPA correcto, relaciones, validaciones Bean Validation, enums.
Repositorios	1,0	Métodos derivados y @Query correctos. Nombres coherentes.
Servicio AutorService + LibroService	1,5	Lógica de negocio, validaciones, excepciones semánticas.
Servicio PrestamoService	1,5	Gestión de stock, estados, detección de vencidos.
Servicio UsuarioService	0,5	Registro, BCrypt, cambio de contraseña.
Controladores (endpoints REST)	1,5	URLs correctas, métodos HTTP, ResponseEntity, @Valid.
Seguridad (Spring Security + JWT)	2,0	Autenticación, autorización por rol, filtro JWT, STATELESS.
Gestión de excepciones + DataLoader	0,5	@RestControllerAdvice, respuestas JSON, datos iniciales.
TOTAL	10,0	

Rúbrica aplicada a cada apartado:

Nivel	%	Descripción
0%	0	No compila, no arranca o no resuelve el apartado.

Nivel	%	Descripción
25%	2,5	Resuelve algorítmicamente pero falla con entradas inválidas.
50%	5	Falla en casos concretos pero el resultado general es válido.
75%	7,5	Correcto pero código no legible o mal estructurado.
100%	10	Cumple todos los requisitos, valida entradas, modular y documentado.

13. Entrega

Comprime la carpeta raíz del proyecto Maven (sin la carpeta **target/**) en un .ZIP:

`Apellido1_Aellido2_Nombre_ProyectoBiblioteca.zip`

Requisitos mínimos para la entrega:

- El proyecto compila con **mvn clean package** sin errores.
- La aplicación arranca con **mvn spring-boot:run**.
- Puedes probar los endpoints con **Postman**, **Insomnia** o la consola H2 en **http://localhost:8080/h2-console**.
- Incluye un archivo **README.md** con instrucciones de arranque y ejemplos de peticiones.

14. Buenas Prácticas y Consejos

- **Inyección de dependencias:** Usa inyección por constructor en lugar de `@Autowired` en campos.
- **Inmutabilidad de entidades:** No expongas directamente las listas (`@OneToMany`) sin motivo.
- **Logging:** Añade `@Slf4j` (Lombok) y registra operaciones importantes con `log.info/warn/error`.
- **Validación doble:** Valida en el controlador (`@Valid`) Y en el servicio (lógica de negocio).
- **Contraseñas:** NUNCA devuelvas el campo password en las respuestas JSON. Usa `@JsonIgnore`.
- **Pruebas:** Escribe al menos tests unitarios para los servicios con JUnit 5 y Mockito (puntos extra).
- **Swagger:** Añade `springdoc-openapi-starter-webmvc-ui` para documentación interactiva (puntos extra).
- **Commits:** Sube tu progreso a un repositorio Git con commits descriptivos (puntos extra).

15. Referencia Rápida de Anotaciones

Anotación	Paquete / Uso
@Entity, @Table	jakarta.persistence – Marca la clase como entidad JPA.
@Id, @GeneratedValue	jakarta.persistence – Clave primaria y estrategia de generación.
@Column(unique=true, ...)	jakarta.persistence – Personaliza la columna en BD.
@ManyToOne, @OneToMany	jakarta.persistence – Relaciones entre entidades.
@JoinColumn	jakarta.persistence – Columna de FK en relaciones.
@Enumerated(EnumType.STRING)	jakarta.persistence – Almacena enum como texto en BD.
@NotBlank, @Size, @Min, @Max	jakarta.validation – Bean Validation en campos.
@Email, @Past, @Future	jakarta.validation – Validaciones de formato.
@Repository	org.springframework – Marca el repositorio (Dao).
@Service	org.springframework – Marca la clase de servicio.
@Transactional	org.springframework – Gestión de transacciones.
@RestController	Spring MVC – Controlador REST (implica @ResponseBody).
@RequestMapping, @GetMapping...	Spring MVC – Mapeo de rutas HTTP a métodos.
@PathVariable, @RequestParam	Spring MVC – Extrae variables de la URL o parámetros.
@RequestBody, @Valid	Spring MVC – Cuerpo JSON + activar Bean Validation.
@RestControllerAdvice	Spring MVC – Manejador global de excepciones.
@ExceptionHandler	Spring MVC – Método que captura una excepción concreta.
@Configuration, @Bean	Spring Core – Clase/método de configuración.
@EnableWebSecurity	Spring Security – Activa la configuración de seguridad.
@PreAuthorize	Spring Security – Seguridad a nivel de método.
@Component	Spring Core – Bean genérico gestionado por Spring.